

Módulo 10 – Capítulo 08 – Sección 01

Data governance para IA: catalogación, clasificación y linaje de datos

El gobierno de sistemas de IA se estructura en tres capas que trabajan de forma integrada, no como dominios independientes. Comprender la relación entre ellas antes de entrar en el detalle de cada una es el prerequisite para que el capítulo tenga coherencia como sistema, y no como una lista de temas relacionados. El **data governance** define qué datos pueden usarse: qué datasets están disponibles, con qué clasificación de sensibilidad, bajo qué restricciones de uso regulatorio, y con qué requisitos de linaje. El **model governance** define qué modelos pueden desplegarse: qué gates de calidad deben superar, qué documentación debe existir, y quién tiene autoridad para aprobar el despliegue. El **RBAC** (Role-Based Access Control) implementa ambos como controles técnicos de acceso: sin el RBAC, las políticas de data governance y model governance son declaraciones de intención, no controles verificables. La relación es jerárquica pero no secuencial: el data governance define los límites del espacio de datos permitido, el model governance define los límites del espacio de modelos permitido, y el RBAC aplica ambos conjuntos de límites como permisos técnicos en los sistemas de la plataforma.

Con este marco claro, el data governance para IA extiende el governance de datos tradicional con tres necesidades específicas que emergen del uso de los datos en contextos de ML. La primera es el **linaje más estricto**: a diferencia del uso de datos en analytics convencional donde un query es un evento puntual, el uso de datos para entrenar un modelo ML crea una relación duradera entre el dato y el artefacto resultante (el modelo), que debe mantenerse trazable mientras el modelo está en producción. La segunda es el **control especial de datos sensibles para entrenamiento**: datos que son aceptables para consultas analíticas bajo el principio de agregación pueden ser problemáticos para entrenamiento si el modelo puede memorizar instancias individuales o si el uso no fue contemplado en el consentimiento original. La tercera es el **riesgo regulatorio específico de IA**: el GDPR, el EU AI Act y regulaciones sectoriales (FINRA, HIPAA) establecen restricciones específicas para el uso de datos en sistemas de IA que no aplican al uso de datos en analytics.

La **catalogación de datos** para IA se implementa con herramientas como Apache Atlas, DataHub (open source de LinkedIn), o Collibra Enterprise. Estas plataformas mantienen un catálogo activo de todos los datasets con sus metadatos: schema con tipos y descripciones de cada campo, propietario (data owner responsable de las decisiones de uso), clasificación de sensibilidad, SLA de frescura, y origen (qué sistemas generan los datos). Para el caso de uso específico de IA, el catálogo añade el grafo de linaje que conecta fuentes de datos → pipelines de procesamiento → datasets de entrenamiento → modelos registrados → endpoints de serving. Este grafo de linaje es la herramienta que permite responder retroactivamente "¿qué modelos en producción usan datos de este sistema?" —una pregunta que emerge cuando un sistema upstream va a ser modificado o deprecado, y que sin el grafo de linaje requiere una investigación manual costosa.

La **clasificación de datos** en el contexto de IA requiere una taxonomía que incluya no solo la sensibilidad del dato en sí (PII, PHI, datos financieros, datos públicos) sino también las restricciones de uso para entrenamiento de modelos. Un dataset de comportamiento de usuarios puede ser `confidential` por contener datos de comportamiento privado, pero dentro de esa clasificación, el uso para entrenar modelos de recomendación puede estar permitido con consentimiento implícito mientras que el uso para entrenar modelos de clasificación de perfiles de usuario puede requerir consentimiento explícito adicional. Esta granularidad de clasificación por tipo de uso es lo que permite que el catálogo de datos sea el punto de control de qué datasets pueden aparecer en qué tipos de experimentos.

Componentes del data governance para IA

- **Data catalog:** inventario centralizado de todos los datasets con schema, owner, classification label (public/internal/confidential/restricted), SLA de frescura, y restricciones de uso para IA; actualizado automáticamente por crawlers del data lake.
- **Classification framework:** taxonomía de sensibilidad de datos con reglas de clasificación automática (detectores de PII, de datos financieros, de datos médicos) y proceso de revisión para uso de datos en modelos; la clasificación es el prerequisite del control de acceso.
- **Lineage graph:** grafo que conecta fuentes de datos → pipelines de procesamiento → datasets de entrenamiento → modelos → endpoints; permite responder "¿qué modelos en producción se ven afectados si deprecamos este dataset?" en segundos.
- **Data access requests:** proceso formal para solicitar acceso a un dataset clasificado para entrenamiento, con registro de la justificación del caso de uso y aprobación del data owner; automatizable para datasets de baja sensibilidad, manual para datos críticos.

- **Retention policies:** reglas de cuánto tiempo se conservan los datasets de entrenamiento (al menos mientras el modelo derivado esté en Production + 30 días), con eliminación automática al expirar y notificación a data owners antes de la eliminación.

Nota del Arquitecto: *El data governance para IA no es burocracia: es el conjunto de controles técnicos que garantizan que los modelos en producción pueden auditarse, que el uso de datos cumple con los compromisos legales con los usuarios, y que los incidentes de datos se pueden investigar con trazabilidad completa. La diferencia entre data governance como proceso burocrático y como control técnico es la automatización: si el acceso a un dataset requiere abrir un ticket manual que tarda tres días en procesarse, el governance se convierte en obstáculo; si el acceso se solicita con un comando que dispara un workflow automático con aprobación del data owner en un portal, el governance se convierte en guardrail.*

El data governance para IA establece las bases sobre las que el model governance puede construirse: sin saber qué datos pueden usarse y con qué restricciones, las políticas de aprobación de modelos no pueden verificar si los datos de entrenamiento usados eran apropiados. La sección siguiente examina el model governance: los procesos y controles técnicos que garantizan que ningún modelo llega a producción sin haber pasado por los gates de evaluación y aprobación correspondientes.

Módulo 10 – Capítulo 08 – Sección 02

Model governance: políticas de aprobación antes del despliegue a producción

El model governance es el conjunto de procesos y controles técnicos que garantizan que ningún modelo llega a producción sin haber pasado por los gates de evaluación y aprobación definidos por la organización. Es, en esencia, el "code review y deployment approval" del ciclo de vida de los modelos de ML: la analogía con el software convencional es precisa porque la motivación subyacente es idéntica —el ser humano que revisa verifica que el artefacto cumple con los estándares de calidad, seguridad y correctitud antes de que llegue a los usuarios. La diferencia fundamental con el software es que en los modelos, los criterios de "correctitud" son estadísticos (métricas sobre umbrales), los riesgos de deployment son probabilísticos (el modelo puede fallar de formas que el código no falla), y los impactos pueden ser de mayor escala si el modelo toma decisiones que afectan a muchos usuarios simultáneamente.

Un proceso de model governance robusto define para cada **categoría de riesgo del modelo** un checklist de aprobación diferenciado. Los modelos de bajo riesgo — recomendaciones de contenido, personalización de UI, autocompletado— pueden tener aprobación automatizada si pasan los gates de calidad técnica: métricas sobre umbral, tests de bias dentro de tolerancia, security scan de la imagen Docker sin vulnerabilidades críticas. Los modelos de riesgo medio —scoring de crédito, personalización de precios, filtrado de contenido— requieren revisión humana de un representante del equipo de ML Engineering más aprobación del equipo de producto que entiende el impacto en el negocio. Los modelos de riesgo alto —decisiones médicas, scoring de riesgo en contextos financieros regulados, sistemas de decisión automatizada sobre contratación— requieren un **Model Review Board** con representación multidisciplinaria: ingeniería, producto, legal, compliance, y para ciertos dominios, ética o representantes de los grupos afectados.

La implementación técnica del model governance se integra directamente en el **model registry** como extensión del flujo de estados. Cuando un modelo se registra en el estado "Staging", el pipeline de CI/CD dispara automáticamente una serie de evaluaciones: **bias audit** con Fairlearn (`MetricFrame`) que mide la disparidad de métricas entre subgrupos

protegidos) o IBM AIF360, security scan del artefacto, performance regression test comparando contra el modelo en Production. Solo cuando todas estas evaluaciones automáticas pasan, el modelo puede avanzar al estado "Under Review" donde se inicia el proceso de solicitud de aprobación humana si es requerida. El audit trail de estas evaluaciones —qué tests corrieron, con qué resultados, cuándo, y por qué el modelo pasó o fue rechazado— se almacena inmutablemente en el registry como metadatos de la versión.

El **model card** es el documento central del model governance: el artefacto que comunica al equipo de revisión y a los equipos consumidores qué hace el modelo, cómo fue creado, qué limitaciones tiene, y qué evaluaciones superó. En una plataforma madura, el model card se genera automáticamente a partir de los metadatos del registry (métricas de evaluación, dataset de entrenamiento, resultados de bias evaluation, linaje de código) con un template estándar que el equipo de governance define. El EU AI Act (en vigor desde 2024) establece requisitos específicos de documentación para sistemas de IA de alto riesgo que un model card bien estructurado satisface directamente, convirtiendo el modelo governance de una best practice en un prerequisite de compliance regulatorio para organizaciones que operan en la Unión Europea.

Aspectos técnicos del model governance

- **Automated gates:** evaluaciones automáticas que el modelo debe pasar antes de poder solicitar aprobación: accuracy sobre threshold, fairness metrics dentro de tolerancia, no vulnerabilidades críticas en el artefacto, latencia SLO cumplida en load test; automatizar el 80% del gate elimina fricciones sin sacrificar rigor.
- **Bias evaluation:** tests con Fairlearn (`MetricFrame`) o IBM AIF360 que miden la disparidad de métricas (accuracy, precision, recall) entre subgrupos definidos (género, edad, geografía, idioma); flag automático con justificación si la disparidad supera el umbral de política.
- **Model card generation:** documento técnico generado automáticamente con metadatos del modelo, métricas de evaluación, limitaciones conocidas, datos de entrenamiento usados, y resultados de bias evaluation; requerido antes de la solicitud de aprobación humana para cualquier categoría de riesgo.
- **Approval workflow:** integración con sistemas de ticketing (Jira, ServiceNow) o pull requests de Git para formalizar la solicitud de aprobación, adjuntar el model card, y mantener registro inmutable de quién aprobó qué y cuándo con qué justificación.
- **Audit trail:** registro inmutable de cada decisión de aprobación y cada resultado de evaluación automática, requerido para demostrar compliance ante reguladores (EU AI

Act, GDPR, FINRA según el sector) y para investigación de incidentes retroactiva.

Nota del Arquitecto: *El model governance efectivo no ralentiza el desarrollo: automatizar la mayor parte posible de los gates (bias evaluation, performance regression, security scan) y reservar la revisión humana solo para modelos de mayor riesgo garantiza que el proceso agrega valor sin convertirse en cuello de botella. La señal de que el governance está mal calibrado es cuando los equipos buscan formas de clasificar sus modelos como "bajo riesgo" para evitar el proceso de revisión: indica que el proceso es percibido como obstáculo, no como garantía de calidad.*

El model governance transforma el despliegue de modelos de un evento informal y variable en un proceso estandarizado, auditable y escalable. La siguiente sección examina cómo el RBAC implementa técnicamente los permisos que tanto el data governance como el model governance definen como políticas: quién puede acceder a qué datos, qué modelos pueden desplegar, y qué operaciones pueden realizar en la plataforma.

Módulo 10 – Capítulo 08 – Sección 03

Control de acceso basado en roles (RBAC) para datos y modelos

El RBAC es la capa de implementación técnica de las políticas definidas en el data governance y el model governance: convierte "solo los miembros del Model Review Board pueden promover modelos de riesgo alto a Production" en una configuración técnica que hace que esa operación sea literalmente imposible para cualquiera que no tenga el rol `model-approver`. Sin RBAC, el data governance y el model governance son políticas escritas en documentos que dependen de la disciplina voluntaria de cada ingeniero para cumplirse; con RBAC, son controles técnicos que se aplican automáticamente en cada operación, independientemente de si el ingeniero está al tanto de la política o no.

El modelo de roles para una plataforma de IA refleja la estructura de responsabilidades que el capítulo ha definido hasta ahora. El rol `data-viewer` puede leer el catálogo de datasets y sus metadatos —nombre, schema, descripción, clasificación de sensibilidad— pero no puede descargar los datos raw. El rol `data-consumer` puede descargar datasets aprobados para entrenamiento previa aprobación del data owner; el proceso de aprobación queda registrado en el audit log del catálogo. El rol `ml-engineer` puede lanzar training jobs sobre datasets aprobados, registrar modelos en el estado Staging en el model registry, y desplegar en ambientes de desarrollo y staging. El rol `model-approver` puede promover modelos de Staging a Production; su pertenencia al Model Review Board se verifica en el IdP corporativo y se replica automáticamente al sistema de permisos del registry via SCIM. El rol `platform-admin` tiene acceso completo y está reservado al equipo de plataforma con acceso auditado y sesiones grabadas.

La implementación técnica del RBAC en una plataforma de IA opera en múltiples capas que deben estar sincronizadas. En **Kubernetes**, RBAC nativo con Roles y RoleBindings por namespace controla qué ServiceAccounts pueden ejecutar pods, crear InferenceServices, o acceder a secretos; los roles de Kubernetes se sincronizan con los grupos del IdP corporativo via SCIM 2.0 para que las altas y bajas de personal se reflejen automáticamente. En el **feature store**, Feast implementa permisos por project y por FeatureView; Hopsworks

implementa permisos por FeatureGroup con roles heredables de la organización. En el **model registry** (MLflow), los permisos se configuran por experiment y por registered model; en Databricks Unity Catalog, los permisos se configuran como políticas de IAM-like sobre catálogos, schemas y tablas. En el **LLM Gateway**, los permisos de acceso por modelo y endpoint se configuran junto con los límites de rate limiting del Capítulo 07.

El **temporary access** y el **just-in-time access** son mecanismos complementarios que abordan el problema del acceso excepcional: cuando un ingeniero necesita acceder temporalmente a datos de producción para investigar un incidente, el acceso permanente de alto privilegio no es la solución correcta. El temporary access grant provisiona el acceso elevado con expiración automática (ej. 4 horas) y registro de todas las operaciones durante ese período en el audit log con una nota de justificación vinculada al ticket del incidente. El just-in-time access, implementado con herramientas como HashiCorp Boundary o CyberArk, va un paso más allá: el acceso privilegiado requiere aprobación de un segundo factor humano y la sesión completa se graba para revisión posterior. Este nivel de rigor es apropiado para accesos a datos de producción de clasificación `restricted` o para operaciones de alto impacto en el registro de producción.

Componentes clave del RBAC para plataformas de IA

- **Roles predefinidos:** conjunto mínimo de roles que cubren las operaciones principales (viewer, developer/ml-engineer, deployer, approver, admin) con principio de mínimo privilegio; el 90% de los usuarios solo necesitan viewer o ml-engineer en sus namespaces de trabajo.
- **Resource-level permissions:** permisos aplicados a instancias específicas de recursos (acceso a un modelo específico en el registry, no a todos los modelos) implementados via resource policies o attribute-based tags en el IdP.
- **Temporary access grants:** mecanismo para solicitar acceso temporal a recursos de mayor privilegio (datos de producción para debugging) con expiración automática y log de todas las operaciones durante el período de acceso.
- **Cross-team access:** proceso formal para que el modelo del equipo A sea consumido por el equipo B via el serving layer, con acuerdo de SLA documentado y visibilidad de uso para el equipo propietario del modelo en el dashboard de la plataforma.
- **SCIM 2.0 synchronization:** sincronización automática entre el IdP corporativo (Okta, Azure AD) y los sistemas de la plataforma para que las altas y bajas de personal se reflejen en los permisos sin intervención manual; las cuentas huérfanas son un riesgo de seguridad crítico.

Nota del Arquitecto: *El RBAC para plataformas de IA debe diseñarse para el caso de uso real, no para el caso ideal. Si los roles son demasiado restrictivos y los ingenieros los bypassen usando cuentas de servicio con permisos excesivos —porque el proceso de solicitud de acceso normal tarda más de lo que el trabajo requiere— el sistema de control de acceso ha fracasado. El equilibrio correcto es que el acceso necesario para el trabajo diario sea conveniente, y el acceso excepcional sea posible pero auditado y justificado.*

El RBAC implementa técnicamente la arquitectura de gobierno que el capítulo ha construido hasta ahora: los límites del data governance y el model governance se materializan como permisos configurables que el sistema aplica automáticamente en cada operación. La siguiente sección examina la auditoría de uso como la capacidad complementaria que permite no solo controlar quién puede hacer qué, sino registrar y consultar retroactivamente quién hizo qué, cuándo, y con qué resultado.

Módulo 10 – Capítulo 08 – Sección 04

Auditoría de uso: quién usó qué modelo con qué datos y cuándo

La auditoría de uso en una plataforma de IA es la capacidad de responder retroactivamente a cualquier pregunta relevante sobre cómo se usaron los modelos y los datos en el pasado. La palabra "cualquier" es ambiciosa pero precisa: cuando un regulador de protección de datos pregunta qué modelos procesaron datos de usuarios europeos en el último trimestre (GDPR), cuando el equipo legal necesita saber si el modelo de decisión crediticia usó atributos protegidos en sus predicciones (ECOA en EE.UU.), o cuando el equipo de ingeniería necesita saber qué ejecutó el training job que generó el modelo que produjo el incidente de producción, la plataforma debe poder responder con evidencia verificable, no con reconstrucciones basadas en memoria.

Esta capacidad requiere instrumentación en tres capas que deben estar conectadas por un identificador de correlación común. La primera capa es el **audit log del LLM Gateway**: quién llamó a qué modelo, con qué prompt (o su hash si el prompt contiene datos sensibles), con qué respuesta, a qué costo, en qué momento. La segunda capa es el **audit log del model registry**: quién registró cada versión de cada modelo, quién la aprobó para producción, cuándo fue desplegada y con qué configuración, quién la deprecó. La tercera capa es el **audit log del sistema de datos**: qué datasets fueron accedidos y descargados, por qué identidad, cuándo, y si fue para entrenamiento o para análisis. El `request_id` o un identificador de sesión de auditoría conecta estos tres logs para investigaciones cross-system: si hay un incidente en el LLM Gateway, el audit log puede conectarse retroactivamente con el log del registry para saber exactamente qué versión del modelo lo causó, y desde ahí al log de datos para saber exactamente qué datos de entrenamiento contribuyeron al comportamiento observado.

La implementación técnica más robusta usa un sistema de **audit logging immutable** como backend: AWS CloudTrail para eventos de recursos AWS, Cloud Audit Logs de GCP para recursos GCP, o un sistema propio sobre S3 con **Object Lock** (modo WORM: Write Once, Read Many) que previene la eliminación o modificación de logs después de su escritura. El

schema de eventos debe ser estandarizado usando algo como CloudEvents o un schema propio con campos: `event_type`, `actor` (human o service account), `actor_id`, `resource_type`, `resource_id`, `action`, `timestamp`, `ip_address`, `result` (success/failure), `metadata` (campos adicionales específicos del evento). Este schema estandarizado es lo que permite queries cross-system eficientes: si todos los eventos usan `actor_id` con el mismo formato que el IdP corporativo, una query por `actor_id=jsmith@company.com` retorna todos los eventos de ese usuario en todos los sistemas de la plataforma.

Las queries de auditoría más comunes deben estar disponibles como **reportes programados** con actualización diaria, no como consultas ad-hoc manuales. El reporte de "uso mensual de LLMs por equipo y proyecto" (para FinOps), el reporte de "modelos desplegados esta semana y sus aprobadores" (para compliance), y el dashboard de "accesos a datos de clasificación restricted en los últimos 7 días" (para seguridad) son ejemplos de reportes que se pre-computan y se publican en el developer portal de la plataforma. Solo las investigaciones forenses no anticipadas —cuya estructura no puede predecirse— requieren acceso directo al sistema de logs raw con un motor de queries como Elasticsearch o BigQuery sobre los logs archivados.

Aspectos técnicos de la auditoría de uso

- **Event schema estándar:** CloudEvents o schema propio con campos `event_type`, `actor`, `actor_id`, `resource_type`, `resource_id`, `action`, `timestamp`, `ip_address`, `result`, `metadata`; la estandarización del schema es el prerequisite de las queries cross-system.
- **Audit log del serving layer:** cada inferencia registra `model_id`, `version`, `input_token_count`, `output_token_count`, `latency`, `caller_identity`, `project_id`; los prompts completos solo si la política de retención y privacidad de la organización lo permite.
- **Cross-system correlation:** `request_id` como identificador universal que conecta el log del LLM Gateway con el log del registry con el log del feature store con el log de la aplicación cliente; sin este identificador, las investigaciones cross-system son manuales e incompletas.
- **Real-time alerts de auditoría:** alertas inmediatas cuando un usuario accede a datos de clasificación `restricted` fuera de su horario habitual, cuando se descarga un dataset de producción desde una IP fuera de la red corporativa, o cuando se intenta una operación de alta privilegio rechazada.

- **Retention y querability:** logs de auditoría retenidos mínimo 7 años para compliance financiero, 5 años para GDPR (con políticas de pseudoanonimización de datos personales dentro del log pasados los 90 días); indexados en Elasticsearch o BigQuery para queries interactivos en segundos sobre años de eventos.

Nota del Arquitecto: *Un audit trail incompleto es tan inútil como ningún audit trail en el momento que se necesita. La efectividad de la auditoría depende de que el 100% de las operaciones relevantes sean registradas de forma confiable, no el 95%. El 5% no registrado es exactamente donde el incidente que se investiga tiene probabilidad de estar: los sistemas bien instrumentados no suelen ser los que producen incidentes.*

La auditoría de uso cierra el sistema de governance haciendo visible retroactivamente lo que ocurrió. La siguiente sección aborda el problema inverso: no qué ocurrió, sino cuánto tiempo deben conservarse los registros de lo que ocurrió —la política de retención y eliminación de modelos y datos de entrenamiento, donde los requisitos de reproducibilidad y el derecho al olvido crean tensiones que requieren resolución técnica explícita.

Módulo 10 – Capítulo 08 – Sección 05

Retención y eliminación: políticas de ciclo de vida para modelos y datos de entrenamiento

Las políticas de retención y eliminación para modelos y datos de entrenamiento presentan una tensión estructural que no tiene equivalente en otros dominios de gestión de datos: el **derecho al olvido** del GDPR —que exige poder eliminar todos los datos de un individuo cuando lo solicita— y la **necesidad de reproducibilidad** de los experimentos de ML —que exige conservar los datos de entrenamiento exactos mientras el modelo derivado esté en producción para poder auditar, reproducir y explicar su comportamiento. Esta tensión no tiene una solución técnica única: requiere una política explícita que defina cómo la organización balancea ambos requisitos, con soporte técnico automatizado para implementarla de forma consistente.

La resolución práctica se alcanza con diferentes estrategias de retención según el tipo de artefacto. Los **logs de inferencia** —prompts y respuestas de producción— tienen la retención más corta: 30-90 días, con eliminación automática al expirar. Estos logs contienen la mayor concentración de PII y datos sensibles del sistema, y su valor para compliance disminuye rápidamente después del período de retención regulatoria mínima. Los **datasets de entrenamiento** se retienen mientras el modelo derivado esté en estado Production en el registry, más 30 días después de que el último modelo que los usó sea archivado: este período provee el tiempo necesario para responder a solicitudes de auditoría o reproducibilidad retroactiva relacionadas con el modelo recientemente archivado. Los **pesos del modelo** se retienen indefinidamente en el estado Archived del model registry —los bytes ocupan espacio en S3 Glacier con costo mínimo— porque la capacidad de cargar y ejecutar un modelo histórico para auditoría forense puede ser necesaria años después de su despliegue. Los **metadatos y el linaje** (hashes, run_ids, commit SHAs, métricas de evaluación) se retienen indefinidamente porque son el coste de almacenamiento más bajo del sistema y el valor de trazabilidad que proveen es permanente.

El **right to erasure** del GDPR cuando se aplica a datos de entrenamiento plantea el problema del **machine unlearning**: si los datos de un usuario específico fueron usados en

el entrenamiento de un modelo, eliminar esos datos del dataset no modifica el modelo ya entrenado. El modelo puede haber memorizado instancias de entrenamiento —fenómeno bien documentado especialmente en LLMs de gran escala— y podría producir outputs que revelan información sobre ese individuo. La solución técnica más robusta actualmente disponible es el **reentrenamiento desde cero** excluyendo los datos del usuario afectado: costoso en tiempo de cómputo, pero la única garantía de eliminación completa de la influencia del dato. La organización debe definir un **SLA de machine unlearning** —típicamente 30 días para completar el proceso de reentrenamiento— y documentarlo en su política de privacidad.

Las **automated lifecycle policies** convierten las políticas de retención de texto en reglas técnicas ejecutables: S3 Lifecycle Rules o GCS Object Lifecycle Management mueven objetos a storage classes más baratos (Glacier en AWS, Coldline en GCP) después de N días de inactividad, y los eliminan automáticamente después de M días. Cuando un modelo cambia al estado Archived en el registry, un webhook dispara automáticamente la configuración de las lifecycle policies para sus datasets de entrenamiento asociados, iniciando el countdown hacia la eliminación sin requerir intervención manual.

Políticas de ciclo de vida técnicas

- **Clasificación de artefactos:** logs de inferencia (retención 30-90 días), datasets de entrenamiento (retención mientras el modelo derivado esté en Production + 30 días post-archivado), pesos del modelo (retención indefinida en Archived), metadatos y linaje (retención indefinida).
- **Automated lifecycle policies:** S3 Lifecycle Rules o GCS Object Lifecycle Management configurados automáticamente cuando un modelo cambia de estado en el registry; la automatización previene que las políticas de retención se omitan por olvido.
- **Right to erasure workflow:** proceso documentado para procesar solicitudes de eliminación de datos de entrenamiento: identificar todos los datasets afectados via el lineage graph del catálogo de datos, ejecutar el pipeline de datos con el dato eliminado, reentrenar el modelo afectado, actualizar el registry con la nueva versión "cleaned", y notificar al solicitante que el proceso está completo.
- **Model archiving:** proceso de pasar un modelo de Production a Archived con registro de la justificación (sustituido por nueva versión, producto discontinuado, requisito legal), manteniendo los pesos disponibles en Glacier/Coldline pero el endpoint desactivado; el archivado no es eliminación.
- **Audit de retención:** reporte mensual de todos los artefactos que están acercándose a su fecha de expiración de política, para que los data owners puedan decidir si extender la

retención (con justificación documentada) o confirmar la eliminación.

Nota del Arquitecto: *La política de retención de artefactos de IA debe ser explícita desde el diseño del sistema, no decidida ad-hoc cuando se necesita espacio en almacenamiento o cuando llega una solicitud GDPR. La eliminación no planificada de un dataset de entrenamiento puede hacer que un modelo en producción no pueda ser auditado —su comportamiento no puede reproducirse si no existen los datos con los que fue entrenado. El principio correcto es: si el modelo está en producción, los datos con los que fue entrenado deben estar disponibles.*

Las políticas de retención y eliminación son el contrapeso necesario al imperativo de trazabilidad del linaje de modelos: no todo puede conservarse indefinidamente, y el sistema debe automatizar tanto la conservación correcta como la eliminación oportuna. La sección de cierre de este capítulo conecta el governance técnico con su consecuencia más importante: la conversión de políticas aspiracionales en controles verificables que escalan con el número de modelos y equipos.

Módulo 10 – Capítulo 08 – Sección 06

Cierre: el gobierno técnico convierte las políticas en controles verificables

El gobierno de datos y modelos en una plataforma de IA tiene sentido como disciplina de ingeniería solo cuando las políticas están implementadas como controles técnicos verificables, no como documentos de política que dependen de que los ingenieros los sigan voluntariamente. La diferencia entre "política de que todos los modelos deben pasar evaluación de bias antes de ir a producción" como documento PDF y como control técnico implementado en el pipeline de CI/CD es la diferencia entre un compromiso aspiracional y una garantía operacional: el control técnico hace que sea físicamente imposible promover un modelo al estado Production en el registry sin que el test de fairness se haya ejecutado y superado. No es que el ingeniero *elija* cumplir la política: el sistema simplemente no permite avanzar sin que esté cumplida.

Esta perspectiva de **policy as code** —las políticas son código ejecutable que el sistema aplica automáticamente— es la que permite a una organización escalar sus prácticas de governance. Con cinco equipos de AI Engineering, un proceso manual de revisión puede funcionar: hay suficiente superficie de comunicación para que las revisiones sean informadas, los revisores conocen el contexto de cada proyecto, y la coordinación humana es manejable. Con cincuenta equipos y cientos de modelos en producción, solo los controles automáticos son escalables: los revisores humanos se convierten en el cuello de botella del proceso, no en el punto de control de calidad. La automatización de los gates no elimina la revisión humana —la reserva para donde agrega más valor, los modelos de mayor riesgo— y libera la capacidad humana de la verificación rutinaria de cumplimiento técnico.

El sistema de governance que este capítulo ha construido —data governance con catálogo y linaje, model governance con gates automatizados y model cards, RBAC con sincronización automática con el IdP, auditoría de uso con logs inmutables, y políticas de retención automatizadas— es mayor que la suma de sus partes. El catálogo de datos alimenta el lineage graph que permite que el model governance verifique si los datos de entrenamiento cumplen las políticas de clasificación. El audit log del registry conectado con el audit log del gateway

provee la trazabilidad completa que los revisores de compliance necesitan. El RBAC garantiza que los procesos de aprobación del model governance no pueden ser bypassados por errores de configuración de permisos.

El EU AI Act ha transformado el gobierno técnico de IA de una best practice a un requisito de compliance para organizaciones que operan en la Unión Europea. Los sistemas de IA de alto riesgo —scoring de crédito, decisiones en empleo, biometría, infraestructura crítica— deben cumplir con requisitos de documentación, trazabilidad de datos de entrenamiento, evaluación de riesgo, y audit logs que son exactamente los componentes que este capítulo ha descrito. Las organizaciones que construyeron su governance técnico de IA antes de que el Act entrara en vigor están en una posición significativamente mejor para demostrar compliance: su infraestructura técnica ya produce la evidencia que los auditores necesitan.

Principio rector

El gobierno técnico efectivo hace que las políticas sean la consecuencia automática de usar la plataforma, no el resultado de la disciplina voluntaria de cada equipo. Cuando cumplir las políticas de governance es el camino de menor resistencia —porque el sistema no ofrece otra opción— el governance ha alcanzado su diseño más efectivo.

"Security is not a product, but a process."

— Bruce Schneier, criptógrafo y experto en seguridad, cuya afirmación sobre la naturaleza de la seguridad aplica con igual fuerza al gobierno de IA: no es un checklist que se completa una vez, sino un sistema continuo de controles técnicos que se operan, evalúan y mejoran en el tiempo.